CPSC 471 Project: Database Normalization Approach

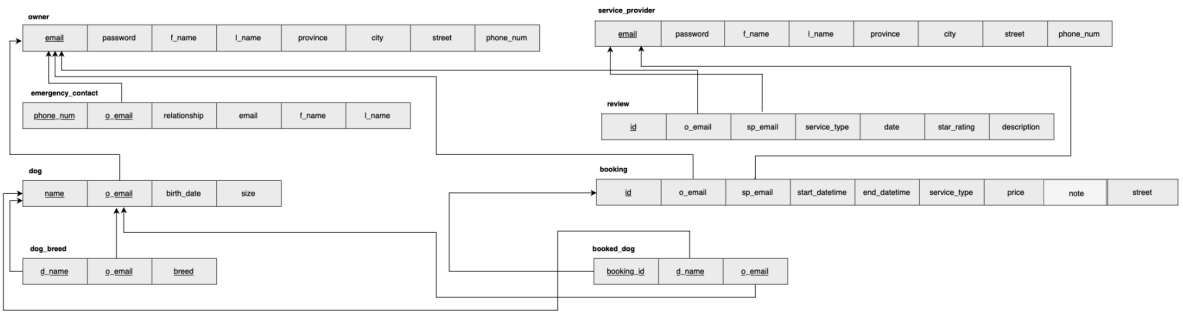
Group T03-4:

1. Braedon Haensel, 30144363
2. Matias Lupick, 30216478
3. Jolene Tan, 30301991

1. Introduction

This report describes the methodology our team used to normalize the database schema for BarkBase. Normalization was essential to ensure that the database is robust, free from redundancy, and resilient to update anomalies.

Below is our ER diagram for reference:



2. Achieving First Normal Form (1NF)

We began by analyzing our data requirements and translating them into an initial, unnormalized schema. As is typical at this stage, the ER model included multi-valued attributes, composite attributes, and several repeating groups. The first step toward normalization involved identifying all relevant attributes and functional dependencies implied by the specification.

To convert this schema into First Normal Form (1NF), we ensured that every attribute in every relation contains only atomic values. Composite attributes such as Address and Name (used in both the Owner and ServiceProvider entities) were decomposed into atomic attributes like `province`, `city`, `street`, `f_name`, and `l_name`. Multi-valued attributes also required restructuring. For example, because a dog may have multiple breeds, we introduced the DogBreed relation with attributes `(d_name, o_email, breed)`, each of which is atomic. Treating all three attributes as part of the primary key allowed us to represent the set of breeds associated with each dog while remaining consistent with 1NF requirements.

3. Achieving Second Normal Form (2NF)

Once the schema satisfied 1NF, we proceeded to Second Normal Form (2NF), which eliminates partial dependencies on composite keys. Many of our relations, such as Owner, ServiceProvider, Review, and Booking, naturally have single-attribute primary keys and therefore satisfy 2NF trivially. The focus, then, was on relations with composite primary keys, including Dog, DogBreed, BookedDog, and EmergencyContact.

In each of these cases, we examined whether any non-key attribute depended only on part of a composite key. For example, in the Dog relation, attributes such as `birth_date` and `size` depend on the entire composite key `(name, o_email)` because a dog's name is only unique within the context of its owner. A similar dependency structure occurs in DogBreed and BookedDog, where all attributes either belong to the primary key or depend fully on it. After evaluating each composite-key relation, we can see that none of them contain partial dependencies, meaning they all satisfy 2NF.

4. Ensuring Third Normal Form (3NF)

Our final objective was to bring the schema into Third Normal Form (3NF) by eliminating transitive dependencies. 3NF requires that non-key attributes depend only on candidate keys and not on other non-key attributes. Throughout most of the schema, this condition was naturally satisfied, since attributes such as names, dates, and service details derive directly from the primary key of their respective tables.

The main challenge arose in the EmergencyContact relation, originally defined as:

```
EmergencyContact(phone_num, o_email, relationship, email, f_name, l_name)
```

At first glance, attributes like `f_name` and `l_name` appear to depend on `email`, which could create a transitive dependency if `email` uniquely identifies a contact person. In that scenario, we would have the dependency chain:

```
phone_num → email → f_name, l_name
```

This would violate 3NF because `email` is not a key in the relation.

However, our group agreed on the semantics that a contact person's identity is not necessarily determined by `email`, rather by the combination of (`phone_num`, `o_email`). Hence `f_name` and `l_name` do not depend on `email` at all, meaning the same owner must have emergency contacts with different contact numbers, but potentially with shared emails as phone number is the more crucial identifier. Under this interpretation, the relation contains no transitive dependencies, and therefore satisfies 3NF.

5. Summary of Normalization Results

Through careful analysis and systematic restructuring, we normalized the initial schema into a clean, well-structured set of relations that meet the requirements of the Third Normal Form. The final design ensures that all non-key attributes depend solely on the keys of their respective relations, eliminating opportunities for update, insertion, and deletion anomalies. The schema effectively models our data requirements while maintaining consistency, reducing redundancy, and supporting long-term scalability and maintainability. Our normalization decisions were guided not only by formal rules but also by the semantic meaning of relationships within the database, ensuring that the resulting schema accurately captures the real-world structure of the system.